

DOM Events

Chapter 1 - Event Propagation

What is Event Propagation?

Before we get into propagation, let's make sure we understand what an event actually is, because everything builds on top of that.

What is an Event? An event is something that happens in the browser - usually triggered by the user or by the browser itself. When you click a button, press a key on the keyboard, move your mouse, or submit a form, the browser creates an event object and sends it out.

Some common events you'll work with:

- **click** - user clicks on an element
- **keydown / keyup** - user presses or releases a key
- **mouseover / mouseout** - mouse enters or leaves an element
- **submit** - user submits a form
- **input** - value inside an input field changes
- **load** - page or a resource finishes loading

Now - what is Event Propagation?

When an event happens on an element in the DOM (for example, you click a button), the browser doesn't just fire the event on that one element and stop. Instead, the event goes on a journey - it travels through the DOM tree, visiting parent and ancestor elements along the way.

This journey is called Event Propagation. The word 'propagate' means 'to spread or travel'. So event propagation literally means: the event spreading/traveling through the DOM.

Real-Life Analogy - The Doorbell

Imagine a house with multiple rooms - a main hall, a living room, and a bedroom inside the living room.

Someone rings the doorbell. Who hears it? Not just the person in the bedroom - everyone in the house hears it! The sound travels outward through all the rooms.

In the same way, when you click a button deep inside the DOM, the event doesn't stay at the button. It travels through all the parent elements surrounding it.

Understanding the DOM Tree

To understand propagation, you need to picture HTML as a tree structure - a family tree of nested elements. Every element (except `<html>`) has a parent, and elements can have children.

Here is a simple HTML structure we will use throughout this entire chapter:

```
<!DOCTYPE html>
<html>
  <body>

    <div id='grandparent'>
      <div id='parent'>
        <button id='child'>Click Me!</button>
      </div>
    </div>

  </body>
</html>
```

If you draw this as a tree, it looks like:

```
window
  |
document
  |
  <html>
    |
    <body>
      |
```

```
<div#grandparent>
  |
  <div#parent>
    |
    <button#child> ← You click here
```

Now, when you click the button, the event does NOT just fire on the button alone. It travels up and down this entire tree. Let's understand exactly how.

The 3 Phases of Event Propagation

Every event in the browser passes through exactly three phases - in this order, every single time:

Phase 1 - Capturing Phase (Top → Down)

The event starts its journey from the very top of the DOM - from the window object - and travels downward through each ancestor element, getting closer and closer to the element that was actually clicked (the target).

Think of it as the browser asking each element from top to bottom: 'Do you want to react to this event before it reaches the target?' If an element has a capturing listener registered, it will fire during this phase.

Important: By default, event listeners do NOT listen during the capturing phase. You have to explicitly opt in (we'll see how later).

Phase 2 - Target Phase (The event arrives!)

The event has reached the exact element that was interacted with. This is called the target element - it's the element the user clicked, typed in, or hovered over.

Event listeners attached to the target element fire during this phase. Both capturing and bubbling listeners on the target itself will run here.

Phase 3 - Bubbling Phase (Bottom → Up)

After the event fires at the target, it starts traveling back up through the DOM tree - from the target element, back through its parents, grandparents, all the way up to window.

This is the default phase. When you write a normal `addEventListener()` without any special options, you are listening in this bubbling phase.

The name 'bubbling' comes from the image of a bubble underwater - it starts at the bottom and rises to the surface, just like the event starts at the target and travels up to window.

Visual Summary of the 3 Phases

PHASE 1 (Capturing): window → document → html → body → grandparent → parent → button

PHASE 2 (Target): The event fires at: button

PHASE 3 (Bubbling): button → parent → grandparent → body → html → document → window

Code Example - Seeing All 3 Phases

Let's write code that proves all 3 phases exist. We'll add listeners to all three elements (grandparent, parent, button) - once for capturing, once for bubbling:

```
const grandparent = document.getElementById('grandparent');
const parent      = document.getElementById('parent');
const child       = document.getElementById('child');

// — CAPTURING listeners (true = capturing phase) —————
grandparent.addEventListener('click', () => {
  console.log('1. Grandparent - CAPTURING');
}, true);

parent.addEventListener('click', () => {
  console.log('2. Parent - CAPTURING');
}, true);

child.addEventListener('click', () => {
  console.log('3. Child - CAPTURING (target reached)');
}, true);

// — BUBBLING listeners (default - no third argument needed) —
grandparent.addEventListener('click', () => {
  console.log('6. Grandparent - BUBBLING');
});
```

```
parent.addEventListener('click', () => {
  console.log('5. Parent - BUBBLING');
});

child.addEventListener('click', () => {
  console.log('4. Child - BUBBLING (target fires again)');
});
```

When you click the button, the console will output in this exact order:

```
// Console Output:
1. Grandparent - CAPTURING      ← Phase 1 starts from top
2. Parent - CAPTURING           ← traveling down...
3. Child - CAPTURING (target reached) ← Phase 2: reached the target
4. Child - BUBBLING (target fires again) ← Phase 3 starts
5. Parent - BUBBLING           ← traveling up...
6. Grandparent - BUBBLING      ← Phase 3 complete
```

Key Takeaway

All capturing listeners fire first (top to bottom).

Then the target's own listeners fire.

Then all bubbling listeners fire (bottom to top).

The numbers in our example (1→6) show the exact sequence.

Chapter 2 - Event Bubbling

What is Event Bubbling?

Event Bubbling is the default propagation behavior in every modern browser. You do not have to configure anything to make it work - it is always on.

Here is the full definition:

Definition

Event Bubbling is the process where, after an event fires on the target element (the element the user interacted with), that same event automatically travels upward through the DOM tree - firing on each ancestor element one by one - until it reaches the very top (the window object).

This means: if you click a button that is inside a div, which is inside another div, the click event will fire on the button first, then on the inner div, then on the outer div, then on the body, then on html, and finally on window.

Why is it called 'Bubbling'?

Imagine you are at the bottom of a swimming pool and you release an air bubble. That bubble rises upward through the water - passing through every layer - until it finally breaks the surface.

An event behaves the same way. It starts at the bottom of the DOM tree (at the target element, which is the deepest / most specific element) and rises upward through every parent element, all the way up to window at the top.

Bubbling in Detail - Step by Step

Let's use a concrete HTML example and walk through it step by step so there is no confusion.

```
<div id='box-1'>
  <div id='box-2'>
    <div id='box-3'>
      <button id='btn'>Click Me!</button>
    </div>
  </div>
</div>
```

Now we add a click listener to every single element:

```
document.getElementById('btn').addEventListener('click', function(e) {
  console.log('Button clicked! e.target.id =', e.target.id);
});
```

```

document.getElementById('box-3').addEventListener('click', function(e)
{
    console.log('Box-3 heard it! e.target.id =', e.target.id);
});

document.getElementById('box-2').addEventListener('click', function(e)
{
    console.log('Box-2 heard it! e.target.id =', e.target.id);
});

document.getElementById('box-1').addEventListener('click', function(e)
{
    console.log('Box-1 heard it! e.target.id =', e.target.id);
});

```

When you click the button, the output is:

```

// Output:
Button clicked! e.target.id = btn      ← fires at the target first
Box-3 heard it! e.target.id = btn      ← bubbles up to box-3
Box-2 heard it! e.target.id = btn      ← bubbles up to box-2
Box-1 heard it! e.target.id = btn      ← bubbles up to box-1

```

Notice something very important: **e.target.id is always 'btn'** - even when the box-3, box-2, and box-1 handlers are running. That's because **e.target** always refers to the element that was **ORIGINALLY** clicked. It never changes during bubbling. (We will explain this more in the next section.)

e.target vs e.currentTarget - The Most Important Distinction

This is one of the most commonly tested concepts in interviews. You must understand both properties clearly.

Property	What it refers to
e.target	The actual element the user clicked / interacted with. This NEVER changes as the event bubbles up.

Property	What it refers to
e.currentTarget	The element whose event listener is currently running. This CHANGES at every level of bubbling.

Here's a code example that shows the difference clearly:

```
<div id='outer'>
  <div id='inner'>
    <button id='btn'>Click</button>
  </div>
</div>

document.getElementById('outer').addEventListener('click', function(e)
{
  console.log('--- Outer handler running ---');
  console.log('e.target.id:      ', e.target.id);          // always
  'btn'
  console.log('e.currentTarget.id: ', e.currentTarget.id); // 'outer'
});

document.getElementById('inner').addEventListener('click', function(e)
{
  console.log('--- Inner handler running ---');
  console.log('e.target.id:      ', e.target.id);          // always
  'btn'
  console.log('e.currentTarget.id: ', e.currentTarget.id); // 'inner'
});

document.getElementById('btn').addEventListener('click', function(e) {
  console.log('--- Button handler running ---');
  console.log('e.target.id:      ', e.target.id);          // 'btn'
  console.log('e.currentTarget.id: ', e.currentTarget.id); // 'btn'
  (same here)
});
```

```
// Output when button is clicked:
--- Button handler running ---
e.target.id:      btn
e.currentTarget.id:  btn

--- Inner handler running ---
e.target.id:      btn    ← still btn! It never changes.
e.currentTarget.id:  inner ← this changes at each level
```

```
--- Outer handler running ---
e.target.id:      btn      ← still btn!
e.currentTarget.id:  outer  ← changed again
```

Simple way to remember this

`e.target` = 'WHO started this event?' - The original clicker. Never changes.

`e.currentTarget` = 'WHO is handling this event right now?' - Changes at every step.

Stopping Bubbling - `stopPropagation()`

Sometimes you want an event to stay on the element it was triggered on and NOT bubble up to parent elements. You can do this by calling `event.stopPropagation()` inside the event handler.

Once `stopPropagation()` is called, the event stops traveling. Parent elements will NOT receive the event.

Real-World Example - Card with a Delete Button

Imagine you have a list of product cards. Clicking anywhere on the card opens the product detail page. But each card also has a small 'Delete' button. If you click Delete, you want to delete the card - NOT open the product page.

Without `stopPropagation`, clicking Delete would ALSO trigger the card's click handler (because of bubbling), causing the product page to open. That's wrong behavior.

```
<div class='card' id='card-1'>
  <h3>iPhone 15</h3>
  <p>Price: ₹79,999</p>
  <button class='delete-btn'>Delete</button>
</div>

// Click on the CARD → open product page
document.getElementById('card-1').addEventListener('click', function()
{
  console.log('Opening product detail page...');
});
```

```
// Click on the DELETE BUTTON → delete the card, do NOT open product
page
document.querySelector('.delete-btn').addEventListener('click',
function(e) {
  e.stopPropagation(); // ← Stops bubbling! Card's handler will NOT
fire.
  console.log('Card deleted!');
  this.closest('.card').remove();
});

// Without stopPropagation - clicking Delete would print BOTH:
// 'Card deleted!'
// 'Opening product detail page...' ← unwanted!

// With stopPropagation - clicking Delete prints ONLY:
// 'Card deleted!' ← correct!
```

stopPropagation vs stopImmediatePropagation

There are two methods that stop event travel, and they behave slightly differently:

Method	What it does
e.stopPropagation()	Stops the event from traveling to PARENT elements. But other listeners on the SAME element will still fire.
e.stopImmediatePropagation()	Stops everything - no parent elements will receive it, AND no other listeners on the SAME element will fire either.

```
// Suppose the button has TWO click listeners:
btn.addEventListener('click', function(e) {
  e.stopPropagation(); // stops bubbling to parents
  console.log('Listener 1 fired');
  // Listener 2 on the same button will still fire!
});

btn.addEventListener('click', function() {
  console.log('Listener 2 fired'); // This still runs!
});

// If you use stopImmediatePropagation() in Listener 1 instead:
```

```
btn.addEventListener('click', function(e) {
  e.stopImmediatePropagation(); // stops everything
  console.log('Listener 1 fired');
  // Listener 2 will NOT fire
});
```

The Power of Bubbling - Event Delegation Pattern

Event Delegation is one of the most important patterns in JavaScript. It is made possible entirely because of event bubbling.

The core idea: Instead of adding a separate event listener to every child element, you add a SINGLE listener on the parent element. When any child is interacted with, the event bubbles up to the parent, and you check `e.target` to figure out which child was actually clicked.

The Problem Without Event Delegation

Imagine a shopping cart with 50 items. Each item has a 'Remove' button. Without delegation, you'd write:

```
// ✗ Bad approach - 50 separate listeners!
const removeButtons = document.querySelectorAll('.remove-btn');

removeButtons.forEach(function(btn) {
  btn.addEventListener('click', function() {
    this.parentElement.remove();
  });
});

// Problems with this approach:
// 1. Creates 50 event listeners in memory - wastes resources
// 2. If you add a new item to the cart dynamically (via JavaScript),
//    the new item's button has NO listener - it won't work!
// 3. More code = harder to maintain
```

The Solution With Event Delegation

```
<ul id='cart'>
  <li class='item' data-id='101'>
```

```

    <span>Nike Shoes</span>
    <button class='remove-btn'>Remove</button>
  </li>
  <li class='item' data-id='102'>
    <span>Adidas T-shirt</span>
    <button class='remove-btn'>Remove</button>
  </li>
  <li class='item' data-id='103'>
    <span>Puma Cap</span>
    <button class='remove-btn'>Remove</button>
  </li>
  // ... imagine 50 items
</ul>

// ✔ Good approach - ONE listener on the parent <ul>
document.getElementById('cart').addEventListener('click', function(e) {

  // Check if the clicked element is a remove button
  if (e.target.classList.contains('remove-btn')) {

    // e.target is the button - .closest() finds the parent <li>
    const item = e.target.closest('.item');
    const itemId = item.dataset.id;

    console.log('Removing item with ID:', itemId);
    item.remove();
  }
});

// This works for ALL current AND future items added dynamically!

```

Event delegation works because when you click the Remove button inside an `<li>`, the click event bubbles up from the button → `<li>` → `<ul>`. The listener on `<ul>` catches it, checks `e.target`, and handles it appropriately.

Benefits of Event Delegation

1. Memory efficient only ONE listener instead of one per element
2. Works for dynamically added elements - even items added to the DOM after the listener was set up
3. Less code - easier to read and maintain
4. Single place to handle all child interactions

Which Events Do NOT Bubble?

Most events bubble, but some important ones do not. You need to know these because if you try to use event delegation with them, it won't work.

Event (does NOT bubble)	Use this instead (bubbles)
focus	focusin
blur	focusout
mouseenter	mouseover
mouseleave	mouseout
load	(none - attach directly to the element)
scroll	(none - attach directly to the scrolling element)

Why does this matter?

If you try to do event delegation using 'focus', it won't work because focus doesn't bubble.

Use 'focusin' instead - it behaves the same but DOES bubble, making delegation possible.

Example: `document.addEventListener('focusin', handler)` ← works for all inputs on page.

Chapter 3 - Event Capturing

What is Event Capturing?

Event Capturing is the first phase of event propagation - the phase that happens BEFORE bubbling. It is the opposite direction: instead of traveling from the target upward (like bubbling), capturing travels from the top downward toward the target.

Definition

Event Capturing (also called the 'trickle down' phase) is the process where the browser starts at the very top of the DOM - at the window object - and travels

DOWNWARD through each ancestor element, moving closer and closer to the target element.

During this downward journey, if any ancestor element has a capturing listener registered for this event type, that listener fires before the event ever reaches the target element.

Capturing is NOT the default behavior. You must explicitly opt in by passing { capture: true } as the third argument to `addEventListener()`.

Real-Life Analogy - Security Checkpoint

Imagine a VIP event venue. Before any guest can reach the main stage (the target element), they must pass through multiple security checkpoints at the entrance, in the lobby, and outside the hall.

These checkpoints happen BEFORE the guest reaches the stage - they can stop the guest early if needed.

Event capturing works the same way: it gives ancestor elements a chance to react to (or even block) an event BEFORE the target element ever sees it.

How to Enable Capturing

The third argument to `addEventListener()` controls which phase you are listening in:

```
// BUBBLING (default) - listens in Phase 3
element.addEventListener('click', handler);           // no 3rd arg =
bubbling
element.addEventListener('click', handler, false);   // false =
bubbling

// CAPTURING - listens in Phase 1
element.addEventListener('click', handler, true);     // true =
capturing
element.addEventListener('click', handler, { capture: true }); // same
thing

// Modern way - options object (recommended for clarity)
element.addEventListener('click', handler, {
```

```

    capture: true,    // listen during capturing phase
    once: true,       // auto-remove listener after it fires once (bonus
feature)
    passive: true,    // tells browser this handler won't call
preventDefault()
  });

```

Capturing vs Bubbling - Complete Side-by-Side Demo

This is the most important example in this chapter. Read through it carefully.

```

<div id='level-1'>
  <div id='level-2'>
    <div id='level-3'>
      <button id='btn'>CLICK</button>
    </div>
  </div>
</div>

```

```

const l1 = document.getElementById('level-1');
const l2 = document.getElementById('level-2');
const l3 = document.getElementById('level-3');
const btn = document.getElementById('btn');

// — CAPTURING LISTENERS (true) —————
l1.addEventListener('click', () => console.log('[CAP] Level-1'),
true);
l2.addEventListener('click', () => console.log('[CAP] Level-2'),
true);
l3.addEventListener('click', () => console.log('[CAP] Level-3'),
true);
btn.addEventListener('click', () => console.log('[CAP] Button'),
true);

// — BUBBLING LISTENERS (default) —————
l1.addEventListener('click', () => console.log('[BUB] Level-1'));
l2.addEventListener('click', () => console.log('[BUB] Level-2'));
l3.addEventListener('click', () => console.log('[BUB] Level-3'));
btn.addEventListener('click', () => console.log('[BUB] Button'));

```

When the button is clicked, here is the exact output:

```
// Output:
[CAP] Level-1      ← Capturing starts at the top, goes down
[CAP] Level-2
[CAP] Level-3
[CAP] Button       ← Target reached (capturing side)
[BUB] Button       ← Target fires bubbling handlers too
[BUB] Level-3      ← Bubbling starts, goes up
[BUB] Level-2
[BUB] Level-1      ← Bubbling reaches the top
```

This output perfectly shows the complete lifecycle of one event: 4 capturing handlers fire first (top to bottom), then 4 bubbling handlers fire (bottom to top).

Stopping an Event During Capturing

You can call `e.stopPropagation()` inside a capturing listener. This will prevent the event from traveling any further - it will NEVER reach the target element or any other elements below.

```
// Scenario: Stop the event at Level-2 during capturing
// The button and Level-3 will NEVER receive this event

12.addEventListener('click', function(e) {
  e.stopPropagation(); // ← Event stops here. Nothing below fires.
  console.log('Event intercepted at Level-2 during CAPTURING');
}, true); // true = capturing phase

// When button is clicked, output is ONLY:
// [CAP] Level-1      ← capturing started
// Event intercepted at Level-2 during CAPTURING ← stopped here
// Nothing else fires - not Level-3, not button, not any bubbling
// handlers
```

Use this with caution!

Stopping propagation during capturing is very aggressive. You are completely preventing the target element from receiving its own event.

Only use this when you intentionally want to block child elements from receiving an event - for example, during a modal overlay where background elements should be unresponsive.

Real-World Use Cases for Capturing

Capturing is uncommon in everyday development. Here are the scenarios where it genuinely makes sense:

1. Blocking interactions during a modal

```
// When a modal is open, block ALL clicks from reaching the page behind it
document.addEventListener('click', function(e) {
  if (modalIsOpen && !modal.contains(e.target)) {
    e.stopPropagation(); // block clicks outside modal
    e.preventDefault();
  }
}, true); // must use capturing to intercept BEFORE target receives it
```

2. Global analytics / click tracking

```
// Track every single click on the page before any handler processes it
window.addEventListener('click', function(e) {
  analytics.track({
    element: e.target.tagName,
    id: e.target.id,
    class: e.target.className,
    timestamp: Date.now()
  });
  // We don't stopPropagation here - we just want to record, not block
}, true); // capturing ensures we run BEFORE any element's own handlers
```

Chapter 4 - Event Traversal

What is Event Traversal?

Event Traversal refers to the technique of navigating through the DOM tree from inside an event handler - moving from the element that was actually clicked to the element you WANT to work with.

Definition

When an event fires, you receive an event object (e). This object gives you e.target - the exact element the user interacted with. But very often, e.target is NOT the element you want to act on.

For example: you click on a small icon inside a <button> inside a inside a . The e.target is the . But you want to find and work with the parent .

Event Traversal is the process of navigating from e.target to the correct ancestor or descendant - using DOM properties and methods like .closest(), .parentElement, .children, and .matches().

Real-Life Analogy - Finding Your Seat's Row

Imagine you're in a cinema. You dropped your ticket (the event target - the exact seat). But you need to find the ROW number (the ancestor element you care about).

Instead of reading the entire cinema layout, you just look at the seat, then look to the side to find the row label - you traverse from the specific (seat) to the general (row).

That's exactly what event traversal does in code.

The Most Important Tool - .closest()

.closest(cssSelector) is the most powerful and most used method for event traversal. Starting from the element it's called on, it walks UP the DOM tree and returns the first ancestor that matches the given CSS selector. If no match is found, it returns null.

Here is a realistic scenario where .closest() is essential:

```
<ul id='task-list'>
  <li class='task-item' data-task-id='1'>
    <span class='task-title'>Buy groceries</span>
    <div class='task-actions'>
      <button class='btn-complete'>✓ Done</button>
      <button class='btn-delete'>✗ Delete</button>
    </div>
  </li>
```

```

<li class='task-item' data-task-id='2'>
  <span class='task-title'>Write assignment</span>
  <div class='task-actions'>
    <button class='btn-complete'>✓ Done</button>
    <button class='btn-delete'>✗ Delete</button>
  </div>
</li>
</ul>

// ONE listener using delegation + traversal
document.getElementById('task-list').addEventListener('click',
function(e) {

  // e.target could be any element inside the <li>
  // We need to find WHICH <li> was interacted with
  const taskItem = e.target.closest('.task-item');

  // If nothing matching '.task-item' found up the tree, stop
  if (!taskItem) return;

  const taskId = taskItem.dataset.taskId;

  // Now check WHICH button was clicked
  if (e.target.classList.contains('btn-complete')) {
    taskItem.style.opacity = '0.5';
    console.log('Task', taskId, 'marked as complete');
  }

  if (e.target.classList.contains('btn-delete')) {
    taskItem.remove();
    console.log('Task', taskId, 'deleted');
  }
});

```

This single event listener handles every button in every task item - current and future ones - using delegation + traversal together. This is the most practical pattern in real frontend development.

Using `.matches()` to Check Element Type

`element.matches(cssSelector)` returns true if the element matches the given CSS selector, and false if it does not. It is useful for checking WHAT was clicked inside a delegated handler.

```

document.addEventListener('click', function(e) {

    // Did the user click any button?
    if (e.target.matches('button')) {
        console.log('Some button was clicked');
    }

    // Did the user click a submit button specifically?
    if (e.target.matches('button[type="submit"]')) {
        console.log('Submit button clicked');
    }

    // Did the user click an element INSIDE a .card?
    // (combine with closest for more power)
    if (e.target.closest('.card')) {
        const card = e.target.closest('.card');
        console.log('Clicked inside card:', card.id);
    }

    // Did the user click a link that has a data-action attribute?
    if (e.target.matches('a[data-action]')) {
        e.preventDefault(); // stop the link from navigating
        const action = e.target.dataset.action;
        handleAction(action);
    }

});

```

Navigating the DOM Tree Manually

Sometimes you need to move to a specific relative element. Here are all the DOM traversal properties you can use:

Property / Method	What it does
<code>e.target.parentElement</code>	Move UP one level to the direct parent element
<code>e.target.closest('.card')</code>	Move UP to the nearest ancestor matching a CSS selector
<code>e.target.children</code>	Get all direct child elements (returns <code>HTMLCollection</code>)
<code>e.target.firstElementChild</code>	Get the first child element

Property / Method	What it does
<code>e.target.lastElementChild</code>	Get the last child element
<code>e.target.nextElementSibling</code>	Get the next sibling element
<code>e.target.previousElementSibling</code>	Get the previous sibling element
<code>e.target.querySelectorAll('li')</code>	Find all matching descendants inside this element

```
// Practical example - navigating relatives from e.target
document.addEventListener('click', function(e) {

  // Example 1: Get the parent
  const parent = e.target.parentElement;
  console.log('Parent:', parent.tagName); // e.g., 'LI'

  // Example 2: Get next sibling
  const next = e.target.nextElementSibling;
  if (next) console.log('Next sibling:', next.className);

  // Example 3: Find all <span> elements inside the clicked element
  const spans = e.target.querySelectorAll('span');
  spans.forEach(span => console.log(span.textContent));

});
```

composedPath() - Inspect the Full Propagation Path

`e.composedPath()` returns an array of ALL elements the event passed through (or will pass through), from the innermost to the outermost. It's very useful for debugging.

```
document.addEventListener('click', function(e) {
  console.log('Full propagation path:');
  e.composedPath().forEach((el, index) => {
    const name = el.nodeName || 'Window';
    const id = el.id ? '#' + el.id : '';
    console.log(`  [${index}] ${name}${id}`);
  });
});

// Clicking a <button id='btn'> inside <div id='wrapper'>:
// [0] BUTTON#btn
```

```
// [1] DIV#wrapper
// [2] BODY
// [3] HTML
// [4] #document
// [5] Window
```

Chapter 5 - Revision, Interview Q&A & Practice

Everything in One Place - Full Revision

Topic	One-Line Summary
Event Propagation	The complete journey of an event through the DOM tree (all 3 phases together)
Capturing Phase	Event travels TOP → DOWN (window to target). Must opt in with { capture: true }
Target Phase	Event arrives at the element you clicked. Both capture and bubble handlers fire here
Bubbling Phase	Event travels BOTTOM → UP (target to window). DEFAULT behavior, always active
e.target	The element that was ORIGINALLY clicked. Never changes during propagation
e.currentTarget	The element whose handler is CURRENTLY running. Changes at every level
stopPropagation()	Stops the event from traveling further. Parent/child elements won't receive it
stopImmediatePropagation()	Stops propagation AND prevents other listeners on the SAME element
preventDefault()	Cancels the browser's DEFAULT action (form submit, link navigate). Unrelated to propagation

Topic	One-Line Summary
Event Delegation	Add ONE listener on parent; use bubbling to handle all child events
.closest(selector)	Walks UP the DOM from current element to find nearest matching ancestor
.matches(selector)	Checks if an element matches a CSS selector. Returns true or false
